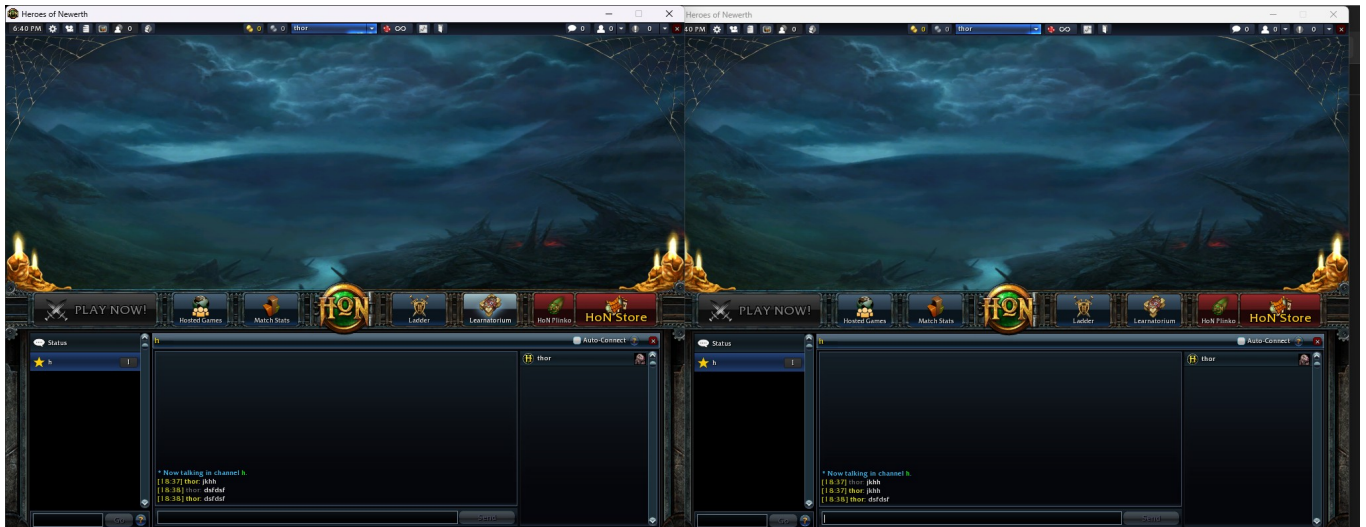


THORGOR

Reconstructing the Heroes of Newerth 3.2.7 Master and Chat Services

A code-level guide to the Python implementations, the reverse-engineering process, the protocol discoveries, and the specific changes made to support multiple local accounts and clients.



Verified end state: two simultaneous HoN clients connected to the custom chat server, sharing a channel and exchanging messages under separate local accounts.

Audience

A computer-science graduate who can read Python, understands basic networking and cryptography, but has not followed the project history.

Prepared from the working source builds: Master Server v24 and Chat Server v8 Multi-Account. This document explains the implementation as it exists, including known limitations and the debugging history that led to the verified result.

Contents

1. Executive overview
2. What was reverse engineered
3. System architecture
4. Development and evidence-driven workflow
5. Master Server v24: source tour
6. HoN-specific SRP authentication
7. HTTP and PHP serialization
8. SQLite account model
9. Master-to-chat identity handoff
10. Chat Server v8: source tour
11. Binary packet framing and parsing
12. Multi-client channel state
13. What changed from the original single-account prototypes
14. Failure history and lessons
15. Running and testing the system
16. LAN and second-node testing
17. Security, correctness, and limitations
18. Suggested next engineering steps
- Appendix A. Key source map
- Appendix B. Protocol and packet reference
- Appendix C. Guided code-reading path

1. Executive overview

The project recreates enough of two discontinued Heroes of Newerth backend services for an unmodified HoN 3.2.7.1 client to authenticate and use shared chat in a private sandbox. The implementation is not a copy of the original server software. It is a clean-room behavioral reimplementaion based on client-side binary analysis, captured requests, controlled experiments, and iterative protocol reconstruction.

Verified capability

Two separate HoN client instances can authenticate as different SQLite-backed accounts, connect to the custom TCP chat service, join the same channel, see the correct roster identities, and exchange messages.

What the two Python services do

Service	Transport	Primary responsibility	Working build
Master server	HTTP on TCP port 80	Two-stage SRP login, account lookup, session proof validation, and issue local chat credentials	v24
Chat server	Custom binary TCP on port 11031	Validate master-issued identity, maintain per-socket user state, manage channels, roster updates, keepalive, and message broadcast	v8 Multi-Account
Shared database	SQLite file	Stores local sandbox accounts and identity metadata used by both services	thorgor_accounts.db

The most important design decision is that the master and chat server share the same account database and derive the same cookie/auth-hash values. That creates a small but coherent local identity system rather than two unrelated mock servers.

2. What was reverse engineered

The work began with the HoN client binary, especially k2.dll, rather than server source code. Ghidra was used to locate strings, callers, hashing routines, packet handlers, and typed response parsers. The goal was not to recover every original symbol; it was to identify the minimum behavior required for client compatibility.

Recovered master-server behavior

- The request names and shapes: f=pre_auth and f=srpAuth.
- The client sends a 2048-bit SRP public value A as 512 hexadecimal characters.
- The server must return salt, server public value B, and salt2 in PHP-serialized form.
- The client returns a 32-byte proof M1; the server must verify it and return M2/HAMK.
- The HoN password preprocessing chain uses pre-MD5 credential material, a fixed salt2 string, two recovered magic strings, MD5, then SHA-256.

- The client expects a typed account payload: several fields must be PHP integers or booleans rather than numeric strings.

Recovered chat behavior

- Binary framing is little-endian: a 16-bit count of bytes after the length field, followed by a 16-bit command and payload.
- AUTH_INFO command 0x0C00 carries account ID, cookie, IP, auth hash, protocol, version, and a platform tail.
- AUTH_ACCEPTED is command 0x1C00 and the correct empty packet is exactly 02 00 00 1c.
- Join-channel and roster packets require a shared channel ID and a complete member snapshot.
- Existing channel members need an incremental joined-user event when a second client enters.
- Channel messages carry account ID, channel ID, and a NUL-terminated UTF-8 string.

3. System architecture

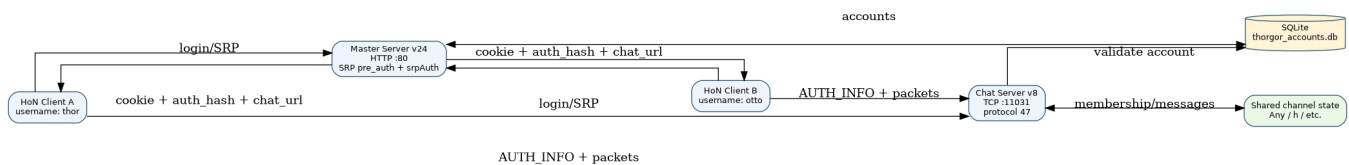


Figure 1. End-to-end architecture of the local HoN sandbox.

The HoN client first speaks HTTP to the master server. After SRP succeeds, the master response includes a local cookie, auth hash, account ID, nickname, and chat address. The client then opens a separate TCP connection to the chat server and presents those credentials in AUTH_INFO. The chat server verifies them against the shared SQLite account record and creates an independent ClientState for that socket.

Trust boundary

The master server proves knowledge of the account password through SRP. The chat server does not repeat SRP; it trusts a deterministic local cookie and auth hash derived from the master account record. This is suitable for a private sandbox, not a hardened Internet service.

4. Development and evidence-driven workflow

The implementation was created as a sequence of experiments. Each experiment used the real HoN client as the compatibility oracle. The working method was:

- Locate a likely client function or string in Ghidra.
- Trace callers and data flow until the network request, hash input, or parser expectation becomes visible.
- Implement the smallest compatible Python response or packet.
- Run the unmodified client and capture logs, request bodies, response bytes, and visible UI behavior.
- Compare the client-supplied proof or packet behavior to the Python server's prediction.
- Freeze known-good versions before changing the next subsystem.

This mattered because the protocol is not merely “standard SRP” or “a normal length-prefixed chat protocol.” Compatibility depends on details such as padding width, exact username bytes, typed PHP fields, endian order, NUL terminators, packet length semantics, and legacy client assumptions.

Artifacts used as evidence

Artifact	What it established
Frozen v19 SRP capture	The pre-MD5 password chain and exact proof match

	were correct.
v24 master logs/captures	The server and client independently calculated the same M1, then the server returned M2.
v7 chat logs	The original server accepted one hardcoded identity but rejected a second account/cookie pair.
v8 two-client screenshot	Separate identities, shared channel state, roster synchronization, and message broadcast all worked.

5. Master Server v24: source tour

Source file: `thorgor_hon_sandboxed_masterserver_v24.py`, 885 lines. The program uses only Python's standard library: `http.server`, `sqlite3`, `hashlib`, `hmac`, `threading`, `secrets`, `urllib.parse`, and supporting modules.

Major components

Component	Lines	Purpose
Config / constants	31-75	Ports, paths, custom SRP group, fixed strings, password-chain mode.
AccountStore	86-201	SQLite initialization and account CRUD.
Serialization and crypto helpers	204-273	PHP serialization, integer padding, SHA-256 helpers, HoN password transform.
Session + Runtime	276-339	Stores one in-progress SRP handshake per client IP and exact username.
<code>create_session()</code>	350-438	Computes all server-side SRP values and expected proofs.
Payload builders	440-493	Builds pre-auth, success, and error response dictionaries.
HTTP Handler	520-744	Parses requests, routes f values, validates proofs, writes captures.
<code>main()</code>	750-885	CLI account management and threaded HTTP server startup.

Threading model

`ThreadingHTTPServer` creates one handler thread per HTTP connection. Shared structures are protected with locks: `AccountStore` uses an `RLock` around database operations, and `Runtime` uses a `Lock` around the temporary handshake dictionary. SQLite connections are opened per operation rather than shared across threads.

```
class Server(ThreadingHTTPServer):
    allow_reuse_address = True
    daemon_threads = True
```

6. HoN-specific SRP authentication

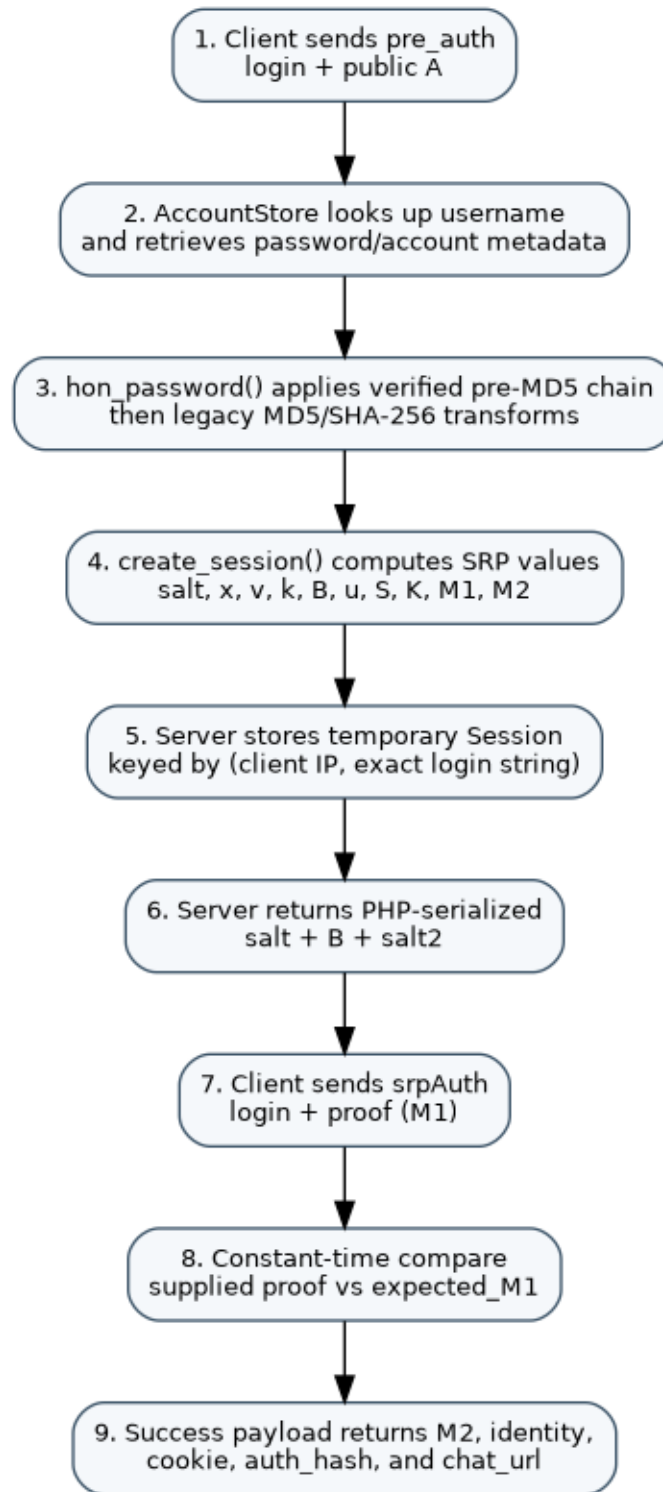


Figure 2. Master-server login sequence implemented by v24.

6.1 Password transformation

The client does not feed the visible password directly into the SRP identity hash. The verified v19 capture showed a pre-MD5 chain. v24 therefore defaults to password_chain="pre-md5".

```

material = hashlib.md5(password.encode("utf-8")).hexdigest()
stage1 = hashlib.md5((material + salt2 + MAGIC1).encode("utf-8")).hexdigest()
transformed = hashlib.sha256((stage1 + MAGIC2).encode("utf-8")).hexdigest()
  
```

This was the decisive fix after earlier multi-account builds used direct mode. A correct database password still produced a wrong verifier if the preprocessing chain differed by even one byte.

6.2 Group and integer encoding

The server embeds a recovered 2048-bit modulus N and generator $g=2$. All critical SRP integers are serialized as exactly 256-byte, big-endian, zero-left-padded values before hashing. This differs from implementations that hash minimal-length integer encodings.

```
WIDTH = 0x100
def pad_num(value: int) -> bytes:
    return value.to_bytes(WIDTH, "big")
```

6.3 Session calculation

For each pre_auth request, create_session() performs the verifier-side SRP calculation. In compact form:

```
inner = H(username || b":" || transformed_password)
x = H(PAD(salt) || inner)
v = g^x mod N
k = H(PAD(N) || PAD(g))
B = (k*v + g^b) mod N
u = H(PAD(A) || PAD(B))
S = (A * v^u)^b mod N
K = H(PAD(S))
M1 = H(H(N) xor H(g), H(username), PAD(salt), PAD(A), PAD(B), K)
M2 = H(PAD(A), M1, K)
```

The client independently calculates the same shared secret and $M1$. The server never receives the password during the wire handshake; it only uses the locally configured password to compute the verifier-side expectation.

6.4 Proof comparison and session consumption

srpAuth locates the temporary Session by (client IP, exact username). It parses the supplied proof, checks that it is 32 bytes, and uses hmac.compare_digest for constant-time comparison. On success, the temporary session is consumed so the same handshake cannot be reused accidentally.

```
matched = (
    len(supplied) == 32
    and hmac.compare_digest(supplied, session.expected_M1)
)
```

Exact login string matters

The database lookup is case-insensitive, but the SRP hash uses the exact username string sent by the client. Earlier revisions substituted the canonical SQLite username, which can change the hash input.

7. HTTP and PHP serialization

The client uses a legacy HTTP endpoint convention and expects PHP serialized data rather than JSON. Handler.handle_all() parses both query-string and application/x-www-form-urlencoded values, then dispatches on parameter f.

```
f=pre_auth&login=<name>&A=<512 hex characters>
f=srpAuth&login=<name>&proof=<64 hex characters>&OSType=...
```

Custom serializer

php_serialize() recursively supports None, booleans, integers, bytes, strings, lists/tuples, and dictionaries. Lengths are byte lengths, not character counts. This is important for UTF-8 strings.

```

if isinstance(value, str):
    raw = value.encode("utf-8")
    return b's:%d:' % len(raw) + raw + b'';

```

Typed account payload

The success response intentionally sends `account_id`, `account_type`, trial-like values, booleans, and thresholds as their expected PHP types. A numeric string can look correct in a log yet fail when the client parser calls `GetInteger()`.

Field	Produced by v24	Purpose
proof	M2 as hex string	Server proof that completes SRP.
account_id	PHP integer	Stable local account identity.
nickname	UTF-8 string	Displayed identity in client/chat.
cookie	THORGOR_LOCAL_COOKIE_##### ##	Credential later sent to chat server.
auth_hash	SHA-1 of local identity material	Second chat credential.
chat_url	127.0.0.1	Address the client uses for chat in the local build.
account_type / standing	PHP integers	Satisfy legacy client account gates.

8. SQLite account model

AccountStore creates and manages a single accounts table. The database is deliberately small and human-manageable.

```

CREATE TABLE IF NOT EXISTS accounts (
    account_id INTEGER PRIMARY KEY AUTOINCREMENT,
    username TEXT NOT NULL UNIQUE COLLATE NOCASE,
    transformed_password TEXT,
    password TEXT,
    nickname TEXT NOT NULL,
    enabled INTEGER NOT NULL DEFAULT 1,
    created_at TEXT NOT NULL DEFAULT CURRENT_TIMESTAMP,
    updated_at TEXT NOT NULL DEFAULT CURRENT_TIMESTAMP
)

```

Account operations

- `add_or_update()`: inserts a new row or resets the password/nickname for an existing username.
- `get()`: performs case-insensitive username lookup and excludes disabled accounts by default.
- `list_accounts()`: supports the management batch file.
- `set_enabled()`: enables or disables login without deleting the identity.
- `delete()`: removes the row.

Security limitation

v24 stores the local sandbox password in plaintext because the server needs the exact password to reproduce the recovered HoN-specific transform during each login. This is acceptable only for disposable test credentials in an isolated environment. A stronger design would store an SRP verifier after fully validating an account-provisioning derivation path.

Why both services use one database

The master server issues identity fields based on `account_id` and `username`. The chat server recomputes and verifies those exact values from the same row. If the two programs point at different database files, valid master logins will be rejected by chat or display stale identities.

9. Master-to-chat identity handoff

After SRP succeeds, v24 returns deterministic local credentials:

```
cookie = f"THORGOR_LOCAL_COOKIE_{account_id:08d}"
auth_hash = SHA1(f"THORGOR_LOCAL_AUTH:{account_id}:{username}")
```

The HoN client includes these fields in the subsequent chat AUTH_INFO packet. v8 does not trust the nickname claimed by the client; it loads the account by `account_id`, recomputes the expected cookie and auth hash, checks enabled status, and then assigns the nickname from SQLite.

Why this fixed the hardcoded-user bug

The original chat prototype accepted only one fixed account/cookie pair and assigned every socket the name `pwnrbwnr`. v8 derives identity from the database row for each authenticated connection.

10. Chat Server v8: source tour

Source file: `thorgor_hon_chatserver_v8.py`, 596 lines. It is a threaded TCP server built with `socketserver.ThreadingMixIn`. Each socket is handled by one `ChatConnection` object, while `ChatWorld` coordinates shared state.

Component	Lines	Purpose
Command constants	46-62	Names recovered numeric packet IDs.
Logging/capture helpers	71-106	Writes readable logs and per-packet JSON capture files.
Framing helpers	109-133	Encodes and extracts length-prefixed packets from a streaming TCP buffer.
<code>parse_auth()</code>	136-187	Parses stable AUTH_INFO fields while preserving unknown platform bytes.
Database discovery/validation	192-275	Finds the v24 DB and reconstructs expected credentials.
ClientState + ChatWorld	278-332	Per-client identity and shared channel registry.
ChatConnection	335-547	Authentication, keepalive, joins, rosters, message broadcast, packet loop.
<code>main()</code>	554-596	CLI, DB selection, threaded TCP startup.

11. Binary packet framing and parsing



Figure 3. HoN chat packet framing used in both directions.

The length field counts every byte after the two-byte length field. Therefore it includes the two-byte command plus payload, but not the length itself.

```
body = struct.pack("<H", command) + payload
packet = struct.pack("<H", len(body)) + body
```

TCP is a byte stream, so one `recv()` can contain half a packet, exactly one packet, or several packets.

`ChatConnection` appends each chunk to `self.buffer` and repeatedly calls `extract_packet()` until no complete packet remains.

Why the original 04 00 auth-accepted packet was misleading

An empty `AUTH_ACCEPTED` packet should contain two bytes after the length field: only the command `0x1C00`. The correct bytes are therefore:

```
02 00 00 1c
```

An earlier `04 00` length could appear to work because the client accidentally consumed two bytes from the next ping packet. This is a classic stream-framing bug: behavior may seem valid until timing or packet coalescing changes.

AUTH_INFO parsing

`parse_auth()` reads only the fields proven stable in protocol 47 and leaves the platform/version remainder as raw bytes. This avoided forcing an older client packet into newer public protocol documentation.

```
uint32 account_id
cstring cookie
cstring ip
cstring auth_hash
uint32 protocol
uint32 client_version_raw
remaining bytes -> platform_tail
```

12. Multi-client channel state

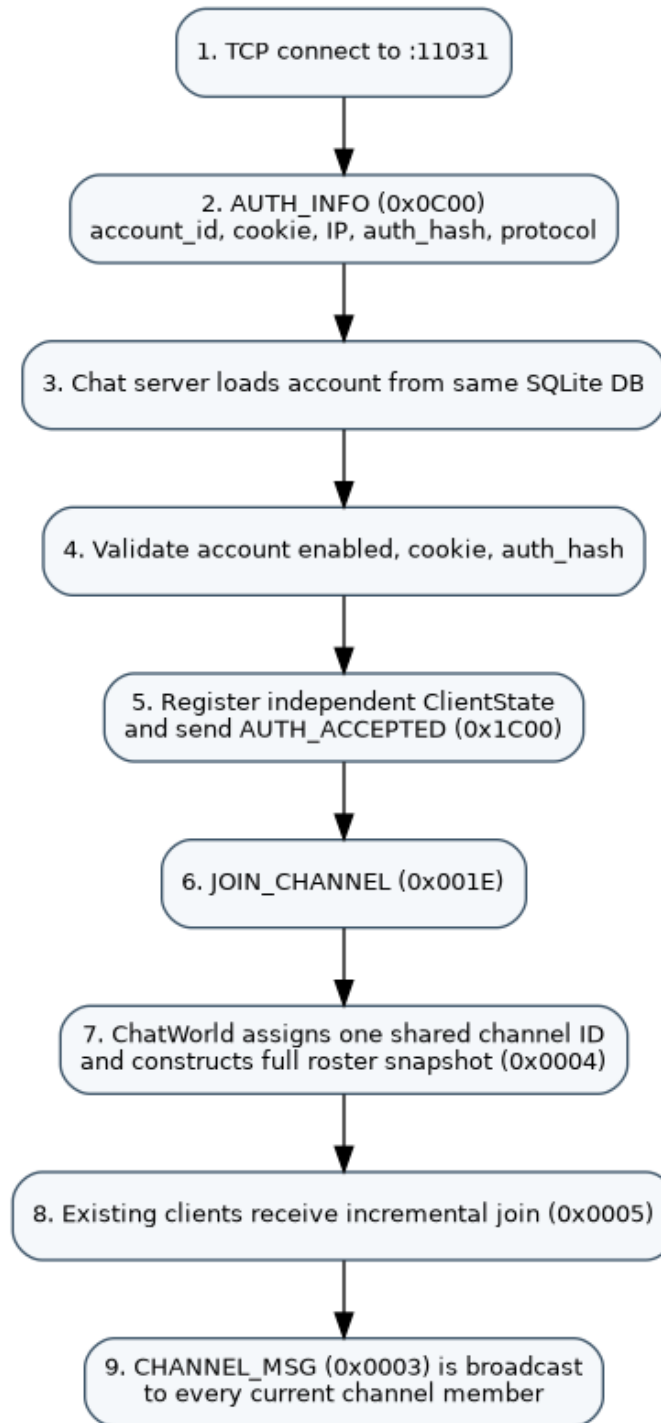


Figure 4. Chat authentication, channel join, and message broadcast sequence.

12.1 One ClientState per socket

ClientState stores handler, account ID, nickname, current channel, and channel ID. Because it belongs to a ChatConnection, two clients on the same IP still remain distinct. ChatWorld indexes them by Python object identity `id(state.handler)`, not by IP address.

12.2 Shared channel IDs

ChatWorld.channel_ids maps channel names to one numeric ID. The first client to create a channel receives the next ID; later clients joining the same name reuse it. This fixed the earlier behavior where every join generated a separate channel universe.

```
channel_id = self.channel_ids.get(channel)
if channel_id is None:
    channel_id = self.next_chat_id
    self.next_chat_id += 1
    self.channel_ids[channel] = channel_id
```

12.3 Complete snapshot plus incremental update

The newly joining client receives command 0x04 with a complete channel snapshot containing the channel name, shared ID, topic fields, and every member record. Existing members receive command 0x05 for the new user so their rosters update without reloading the whole channel.

12.4 Message broadcast

When a client sends command 0x03, the server parses message then channel ID. It constructs the server form as account ID + channel ID + NUL-terminated message and sends it to every current member, including the sender. This is why both windows display the same message history.

```
response = struct.pack("<II", account_id, channel_id) + cstr(message)
for member in WORLD.channel_members(channel):
    member.handler.send_packet(HON_SC_CHANNEL_MSG, response)
```

12.5 Keepalive and presence

A background heartbeat sends command 0x2A00. The client replies with 0x2A01. After authentication, client command 0x00B9 triggers a status update packet 0x0066 with online status 0 and an empty NUL-terminated status string.

13. What changed from the original single-account prototypes

Area	Original behavior	Final behavior
Master credentials	One global --password value; arbitrary username effectively shared it.	SQLite lookup selects an enabled account and its own password.
SRP password path	Working v19 used pre-MD5; early multi-account revisions accidentally defaulted to direct.	v24 defaults and launches explicitly with pre-md5.
Username hashing	Some revisions substituted canonical DB username.	v24 hashes the exact login string sent by HoN.
Chat authentication	Hardcoded account/cookie pair.	Loads account by ID and validates deterministic cookie + auth hash.
Displayed name	Every connection became pwnrbwnr.	Nickname comes from the authenticated database row.
Channel identity	Potentially new channel ID per joining client.	One channel ID per channel name.
Roster update	Second user could disconnect or remain invisible.	New user receives full snapshot; existing users receive 0x05 join

		event.
Concurrency	Single successful path.	Independent threaded connections and shared locked world state.

14. Failure history and lessons

The failed intermediate builds were useful because each isolated a compatibility assumption. The final design should be understood through these lessons rather than treated as magically correct code.

Revision/problem	Observed symptom	Root cause / lesson
v20 multi-account	All configured accounts produced SRP proof mismatch.	Credential storage changed the known-good transformation path; cryptographic byte paths should remain frozen while adding persistence.
v21	Client displayed Session.__init__ unexpected keyword transformed_password.	Dataclass field and constructor keyword diverged; a simple integration error prevented the handshake before cryptography.
v22/v23	Constructor fixed, but proof still mismatched.	The build still differed from the actual successful baseline; line-by-line or capture-based comparison beats conceptual similarity.
v24 discovery	Frozen v19 capture showed pre-md5 while new builds printed direct.	Startup configuration is part of protocol correctness. A valid algorithm with the wrong mode is invalid.
Chat v7 second user	AUTH REJECTED LOCALLY account/cookie mismatch; hardcoded pwnrbwnr.	The chat service needed to consume the master's real account model, not a single test constant.
Early auth packet length	Connection seemed partially alive despite wrong length.	TCP packet boundaries are not message boundaries; malformed framing can borrow bytes from the next packet.

Engineering lesson

When reverse engineering a protocol, preserve a frozen known-good build and capture. Make one behavioral change at a time, and compare bytes, not intentions.

15. Running and testing the system

15.1 File placement

Keep the master and chat folders separate if desired, but make both reference exactly one `thorgor_accounts.db`. The simplest layout is:

```
C:\IP\thorgor\master\thorgor_hon_sandboxed_masterserver_v24.py
C:\IP\thorgor\master\thorgor_accounts.db
C:\IP\thorgor\chat\thorgor_hon_chatserver_v8.py
```

15.2 Add accounts

```
python thorgor_hon_sandboxed_masterserver_v24.py --database thorgor_accounts.db --add-account thor <password> --nickname thor
python thorgor_hon_sandboxed_masterserver_v24.py --database thorgor_accounts.db --add-account otto <password> --nickname otto
python thorgor_hon_sandboxed_masterserver_v24.py --database thorgor_accounts.db --list-accounts
```

15.3 Start master

```
python thorgor_hon_sandboxed_masterserver_v24.py --host 0.0.0.0 --port 80 --database C:\IP\thorgor\master\thorgor_accounts.db --password-chain pre-md5
```

15.4 Start chat

```
python thorgor_hon_chatserver_v8.py --host 0.0.0.0 --port 11031 --db C:\IP\thorgor\master\thorgor_accounts.db
```

15.5 Success criteria

- Master startup prints Password chain: pre-md5.
- Master SRP_AUTH log prints MATCH=True for each correct account/password.
- Chat log prints AUTH ACCEPTED with the expected account and nickname.
- Two clients can join the same channel and see member count 2.
- A message typed in either client appears in both clients under the correct sender identity.

16. LAN and second-node testing

Both services bind to 0.0.0.0 by default, which permits LAN connections. The second machine must resolve the HoN master hostname to the server machine's LAN IP, and the firewall must permit inbound TCP 80 and 11031.

Checklist

7. Give the server PC a stable private IP, for example 192.168.1.50.
8. Start master and chat with `--host 0.0.0.0`.
9. Point the second node's HoN master hostname to 192.168.1.50 using the hosts file or sandbox DNS.
10. Allow inbound TCP 80 and 11031 on the server PC.
11. Ensure the master response advertises a chat address reachable from the second node. A hardcoded 127.0.0.1 makes the remote client connect to itself.
12. Test connectivity with Test-NetConnection before launching HoN.

```
Test-NetConnection 192.168.1.50 -Port 80
Test-NetConnection 192.168.1.50 -Port 11031
```

Current LAN caveat

The v24 success payload shown in the working source sets `chat_url` to 127.0.0.1. That works for two instances on one computer but must be made configurable or changed to the server LAN IP for a physically separate node.

17. Security, correctness, and limitations

Appropriate scope

This code is designed for an isolated learning and interoperability sandbox. It is not production-ready authentication infrastructure.

Known limitations

- Passwords are stored in plaintext in SQLite.
- The master-issued cookie and auth hash are deterministic and not expiring bearer tokens.
- Chat authentication does not cryptographically bind the chat socket to the earlier SRP session.
- HTTP and chat traffic are unencrypted.
- Session Runtime is in-memory only and keyed by IP + username, which may have edge cases behind NAT or with overlapping attempts.
- The master success payload contains hardcoded local values such as chat_url and ip.
- Chat implements only a subset of commands; unsupported packets are logged as UNKNOWN.
- Channel leave/disconnect broadcasting is incomplete; unregister removes state but does not yet send a full 0x06 event to peers.
- No persistent chat history, clans, friends, moderation, matchmaking, or game-server orchestration is implemented.

Correctness strengths

- Cryptographic proof comparison uses hmac.compare_digest.
- SRP private values and salts use secrets.randbits.
- SQLite queries use parameter substitution rather than string interpolation.
- Shared mutable structures use locks.
- Packet captures preserve raw hex and ASCII for future reverse engineering.
- Unknown platform-tail bytes are preserved rather than over-parsed.

18. Suggested next engineering steps

Priority	Improvement	Reason
1	Make advertised master IP/chat_url configurable	Required for a true second physical node.
2	Issue random short-lived chat tokens stored in a sessions table	Binds chat login to a successful master login and removes deterministic credentials.
3	Store SRP verifier instead of plaintext password	Improves credential storage after the exact provisioning derivation is validated.
4	Add explicit leave/disconnect roster packets	Keeps all clients synchronized when a user exits.
5	Add integration tests with recorded packets	Prevents regressions in framing, serialization, and field typing.
6	Create a protocol constants module shared by tools/tests	Reduces duplicated magic values and documentation drift.
7	Add structured logging and	Makes analysis easier without

	capture redaction	leaking test passwords or secrets.
8	Implement configurable channels and admin diagnostics	Supports broader sandbox experiments.

Recommended test suite

- Unit tests for PHP serialization of every used type.
- Golden-vector tests for hon_password(), x, B, M1, and M2 using the verified v19 capture.
- Framing tests for partial packets, multiple packets in one buffer, invalid lengths, and exact auth-accepted bytes.
- Database tests for case-insensitive usernames, disabled accounts, nickname changes, and two services pointing at the same file.
- End-to-end test that launches both servers, simulates two clients, joins one channel, and checks broadcast delivery.

Appendix A. Key source map

Source location	Read this to understand
Master 86-201	Account schema, CRUD, and password storage.
Master 204-273	PHP serialization, integer byte encoding, and HoN password transform.
Master 298-339	Temporary SRP session lifetime and lookup key.
Master 350-438	Core verifier-side SRP implementation.
Master 440-488	Fields returned to the client after pre-auth and final authentication.
Master 520-744	HTTP parsing, routing, capture creation, and proof validation.
Chat 109-133	Exact binary framing.
Chat 136-187	AUTH_INFO parser.
Chat 200-275	Database discovery and credential reconstruction.
Chat 278-332	Per-client and shared channel state.
Chat 335-405	Socket authentication and heartbeat startup.
Chat 407-488	Channel snapshot, incremental join, and message broadcast.
Chat 490-547	Command dispatch and streaming TCP receive loop.

Appendix B. Protocol and packet reference

Master HTTP requests

Stage	Client fields	Server fields
-------	---------------	---------------

pre_auth	f, login, A	salt, B, salt2, vested_threshold, success marker
srpAuth	f, login, proof, OS/version values	proof=M2, account identity, cookie, auth_hash, chat_url, typed defaults

Chat commands implemented

Direction	Command	Meaning
Client -> Server	0x0C00	AUTH_INFO
Server -> Client	0x1C00	AUTH_ACCEPTED
Server -> Client	0x2A00	PING
Client -> Server	0x2A01	PONG
Client -> Server	0x00B9	Post-auth presence request
Server -> Client	0x0066	Status update
Client -> Server	0x001E	Join channel
Server -> Client	0x0004	Complete changed-channel snapshot
Server -> Client	0x0005	Incremental member joined
Client <-> Server	0x0003	Channel message
Client -> Server	0x0022	Leave channel (only logged currently)

String and integer encoding

- Chat integers are little-endian on the wire.
- Chat strings in the implemented packets are UTF-8 followed by one NUL byte.
- SRP integers are big-endian and zero-left-padded to 256 bytes before hashing.
- Master HTTP values are form/query text; responses are PHP serialized byte strings.

Appendix C. Guided code-reading path

A CS graduate can become productive fastest by reading the code in this order:

13. Read master Config, constants, AccountStore, and success_payload() to understand the identity model.
14. Read hon_password() and create_session() with the SRP equations beside the code.
15. Read Handler.handle_preauth() and handle_srp_auth() to connect HTTP requests to Session state.
16. Read chat_encode_packet()/extract_packet() and manually decode 02 00 00 1c.
17. Read parse_auth(), load_account(), expected_cookie(), and expected_auth_hash() to understand service-to-service trust.
18. Read ChatWorld.join(), then ChatConnection.join_channel() and channel_message() to understand the successful two-client screenshot.
19. Finally inspect the capture/log functions. They are the instrumentation that made the reverse engineering iterative rather than speculative.

Closing summary

The central achievement is not simply that two Python scripts run. The project established a repeatable method for turning opaque legacy-client behavior into a working compatibility layer: identify client expectations in Ghidra, reconstruct byte-accurate protocol operations, instrument every exchange, preserve known-good evidence, and iteratively replace hardcoded test assumptions with shared state and multi-client logic.

Current result

Master Server v24 and Chat Server v8 form a coherent local HoN identity and chat environment: independent accounts, verified SRP login, authenticated chat handoff, correct per-client identities, shared channels, synchronized rosters, and message broadcast.

Source basis: ThorGor HoN Sandboxed Masterserver v24 and ThorGor HoN LAN Chat Server v8 Multi-Account, as provided and verified on July 10, 2026.